Exploratory Projects (Exploring cutting-edge technologies)

Exploratory projects focus on **cutting-edge topics** in Big Data Systems and AI. These projects are research-oriented and involve optimizing system performance or applying novel techniques to large-scale data. There are three topics available (e.g., **Topic A, B, C**). You only need to select **one topic** to address.

Topic A: Scalable GraphRAG - Knowledge-Enhanced Generation with Graph Analytics

- **Description:** While traditional RAG relies on vector similarity, it often fails to capture multi-hop reasoning and structural relationships. This project aims to build a **Hybrid Retrieval System** that combines Big Data Graph Analytics with LLMs. You will engineer a pipeline that scales knowledge graph construction using Apache Spark and utilizes graph algorithms to enhance context retrieval quality.
- **Key Tasks:**
    1. **Distributed Graph Construction:** Process a large-scale unstructured text corpus (e.g., Wikipedia subset) using **Apache Spark**. Implement parallel information extraction to identify entities and relations, constructing a massive Knowledge Graph stored in DataFrames or RDDs.
    2. **Graph Analytics & Indexing:** Apply graph algorithms (e.g., **PageRank**, **LPA/Louvain**) using **Spark GraphX** or **GraphFrames**. Enrich the graph data with these topological scores to prioritize influential entities during retrieval.
    3. **Hybrid Retrieval & Generation:** Implement a retrieval mechanism that queries both the graph and vector database. Feed the structured graph context into an LLM to answer complex queries, comparing the performance against a baseline naive RAG system.
- **Recommended Stack:** Apache Spark (GraphX/GraphFrames), Neo4j (or NetworkX for prototyping), LangChain, Sentence-Transformers.
- **Recommended Dataset:** Wikipedia Dump, DBPedia subset, Financial News Dataset.

Topic B: High-Performance Inference System & Latency Optimization

- **Description:** This project focuses on High-Throughput Batch Processing of Large Language Models using Big Data frameworks. You will build a distributed pipeline that ingests a massive dataset of prompts (e.g., ShareGPT), uses Apache Spark to distribute the inference workload across a cluster, and performs post-inference analytics on the generated text. The goal is to maximize tokens-per-second (Throughput) and minimizing latency.
- **Key Tasks:**
    1. **Scalable Inference Engine Setup:** Deploy a state-of-the-art inference server (e.g., vLLM or TGI) capable of handling concurrent requests. Configure advanced features like **Tensor Parallelism** or **Continuous Batching** to utilize GPU resources efficiently.
    2. **Dataset Preprocessing with MapReduce/Spark:** Ingest a large-scale

conversation dataset (e.g., ShareGPT) into HDFS or Object Storage. Develop a Spark job to clean, dedup, and tokenize the raw text. Use Spark RDDs or Spark SQL to organize data into optimal batch sizes, ensuring balanced workload distribution across the cluster.

3. **Workload Simulation & Profiling:** Instead of random requests, process a real-world conversation dataset (e.g., ShareGPT) to simulate realistic prompt-length distributions. Build a client to generate high-concurrency traffic and profile system bottlenecks (e.g., KV Cache usage, prefill vs. decode latency).
4. **Optimization Implementation:** Analyze the performance logs. Propose and implement a system-level optimization, such as **Prefix Caching**, customized **Request Scheduling Policies**, or Speculative Decoding.

- **Recommended Stack:** Docker/Kubernetes, vLLM/SGLang, Prometheus & Grafana (for observability), Python (AsyncIO/Ray).
- **Recommended Dataset:** ShareGPT Dataset, Alpaca Evaluation Dataset.

Topic C: Large-Scale Distributed LLM Fine-Tuning

- **Description:** This project explores the intersection of Big Data engineering and AI training. It requires building a hybrid pipeline that utilizes **Apache Spark** for large-scale data preprocessing and **Distributed Training frameworks** to fine-tune Transformer models (e.g., BERT, Llama, T5) on multi-GPU clusters.
- **Key Tasks:**
    1. **Preprocessing with Spark:** Preprocess the dataset by cleaning the text (e.g., removing stopwords) and tokenizing it into input sequences compatible with transformer models (e.g., BERT format). Use Spark RDDs or Spark SQL to parallelize these preprocessing tasks for large datasets.
    2. **Distributed Training:** Load a pre-trained model using Hugging Face Transformers and distribute training across multiple GPUs with PyTorch Distributed or Horovod. Implement training loops with gradient accumulation to handle memory constraints during training.
    3. **Model Evaluation and Real-Time Inference:** Evaluate the fine-tuned model on the test dataset (e.g., accuracy for classification). Record performance metrics (e.g., GPU utilization, training time, accuracy).
- **Recommended Stack:** Apache Spark, Hugging Face Transformers, PyTorch Distributed / Horovod, CUDA GPUs.
- **Recommended Dataset:** Wikipedia Dump Data, IMDB Movie Reviews Dataset, Amazon Product Reviews Dataset.

Requirements for exploratory projects：

- The proposal of the project should contain the following contents:
    1. Description of the task you want to perform.
    2. Description of the data sets you plan to use.
    3. Brief process for implementing your project.

4. What technologies you plan to use.

- The presentation of this project should contain the following contents:
  1. Description of the task.
  2. Description of the data sets you used.
  3. Model training pipeline showing all the technologies used and their relationships.
  4. How you implemented the project in Spark, including any optimizations that you have done.
  5. Results obtained, or a demo if your project is providing a service.
  6. A discussion on the benefits of using these big data technologies over traditional methods.
- The final report should contain all the above contents in the presentation, as well as the following additional materials:
  1. The source code of your implementation (as a separate file).
  2. References.
- Exploratory projects will be graded based on the following criteria (by the order of importance):
  1. System or pipeline complexity and the synergy of different technologies.
  2. The integration of different data types (structured/semi-structured/unstructured, batch vs streaming).
  3. Rationality of method design.
  4. Scalability of the implementation.
  5. Performance and accuracy.